Topic 5: Generators

Each problem is worth 2 points (one point per printed line).
Generators produce values on demand rather than all at once. This topic covers the `yield` syntax, generator comprehensions, lazy built-ins (`map`, `filter`, `zip`, `enumerate`, `itertools`), hand-written iterator classes, and generator-based implementations of familiar containers.

1. Yield Syntax

A function that contains `yield` is a *generator function*. Calling it does not run the body; it returns a generator object. The body executes incrementally, yielding one value per `next()` call and raising `StopIteration` when the function returns.

1.1 Basics

Key points:

- A generator object can be iterated only once. A second `list()` on the same object sees an empty sequence.
- A missing `yield` makes the function return `None`, so `list()` on the result raises `TypeError`.
- `return` inside a generator ends it early; subsequent `yield`s are never reached.
- `next()` on an exhausted generator raises `StopIteration`.

Problem 1. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    print('list(g())=', list(g()))
    print('sum(g())=', sum(g()))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 2. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i
try:
    print('list(countdown(5))=', list(countdown(5)))
    print('sum(countdown(5))=', sum(countdown(5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 3. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    it = g()
    print('list(it)=', list(it))
    print('sum(it)=', sum(it))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 4. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    it = g()
    print('next(it)=', next(it))
    print('list(it)=', list(it))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 5. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i
try:
    print('list(countdown(3))=', list(countdown(3)))
    print('next(countdown(0))=', next(countdown(0)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

1.2 Infinite Generators

Because generators produce values on demand, they can describe infinite sequences. A generator whose body is `while True: yield ...` never raises `StopIteration`.
To use an infinite generator, consume it lazily: call `next()` for a finite number of values, or wrap it in a finite combinator such as `itertools.islice`. Calling `list()` on an infinite generator runs forever and raises `MemoryError`. Fibonacci numbers are the classic infinite generator: two state variables `a, b` are advanced on each iteration with the simultaneous assignment `a, b = b, a + b`.

Problem 6. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    it = nats()
    print('next(it)=', next(it))
    print('next(it)=', next(it))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 7. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        i += 1
        yield i
try:
    it = nats()
    print('next(it)=', next(it))
    print('next(it)=', next(it))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 8. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('sum=', sum(nats()))
    print('list=', list(nats()))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 6 / 16 = 0.38

1.3 Generator Comprehensions

A generator comprehension `(expr for x in xs)` is a concise generator expression. Like a generator function, it is lazy and one-shot: the same comprehension object can be iterated only once.
The loop variable is captured by reference (late binding). If the comprehension references an outer variable that changes after the comprehension is built, the change is visible when the comprehension is consumed.
Nested comprehensions `(y for x in xs for y in x)` flatten one level of structure, applying the inner `for` at each outer iteration.

Problem 9. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
try:
    print('list(xx)=', list(x * x for x in xs))
    print('sum(xx)=', sum(x * x for x in xs))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 10. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
n = 2
gen = (n * x for x in xs)
n = 3
try:
    print('n now=', n)
    print('list(gen)=', list(gen))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 11. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [[1, 2], [3, 4], [5]]
try:
    print('list=', list(y for x in xs for y in x))
    print('sum=', sum(y for x in xs for y in x))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 12. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('start')
    print('list=', list(x for x in nats()))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

2. Laziness

A lazy consumer pulls values from an iterator only when needed. This section covers user-written wrappers (`take`), the built-in lazy consumers (`zip`, `map`, `filter`, `enumerate`), the `itertools` module, and pipelines that chain them together.

2.1 Take: Generator Wrapping a Generator

`take(g, n)` is a generator that pulls exactly `n` values from another generator `g`. Because both are generators, state (the position within `g`) is shared: consuming through `take` also advances `g`.
If `g` runs out before `n` items are taken, `next(g)` raises `StopIteration`, which propagates out of `take`.

Problem 13. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    print('list(t)=', list(take(nats(), 5)))
    print('sum(t)=', sum(take(nats(), 4)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 14. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n - 1):
        yield next(g)
try:
    print('list(t)=', list(take(nats(), 5)))
    print('sum(t)=', sum(take(nats(), 4)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 15. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    g = nats()
    print('take=', list(take(g, 3)))
    print('next(g)=', next(g))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 16. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n):
        yield next(g)
        next(g)
try:
    print('list(t)=', list(take(nats(), 4)))
    print('sum(t)=', sum(take(nats(), 3)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 17. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def small():
    yield 'a'
    yield 'b'

def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    print('take2=', list(take(small(), 2)))
    print('take3=', list(take(small(), 3)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

2.2 Lazy Built-ins

`zip`, `map`, `filter`, and `enumerate` return lazy iterators, not lists. `zip` stops at the shortest input; `map` stops at the shortest input when given more than one iterable. All of them can be forced to a list with `list()`.
Subtlety: when a generator is passed to two arguments of the same lazy consumer (e.g. `zip(g, g)`), each argument pulls from the same underlying iterator, so the two output positions are interleaved.
`filter(None, xs)` keeps only truthy elements; `0`, `''`, `[]`, `None`, and `False` are all dropped.

Problem 18. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
ys = ['a', 'b', 'c']
try:
    print('zip(xs, ys)=', list(zip(xs, ys)))
    print('zip(ys, xs)=', list(zip(ys, xs)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 19. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
    yield 4
try:
    it1 = g()
    print('zip2=', list(zip(it1, it1)))
    it2 = g()
    print('zip3=', list(zip(it2, it2, it2)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 20. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 'a'
    yield 'b'
    yield 'c'
try:
    it = g()
    print('enum=', list(enumerate(it)))
    print('again=', list(enumerate(it)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 21. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
ys = [10, 20, 30]
try:
    print('map=', list(map(lambda a, b: a + b, xs, ys)))
    print('map2=', list(map(lambda a, b: a * b, xs, ys)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 22. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [0, 1, '', 'a', [], [1], None, False]
try:
    print('filter=', list(filter(None, xs)))
    print('count=', len(list(filter(None, xs))))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 23. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('start')
    print('list=', list(map(lambda x: x * x, nats())))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

2.3 itertools

The `itertools` module provides efficient lazy building blocks.

- `count(start, step)` — infinite arithmetic sequence.
- `cycle(xs)` — repeats `xs` forever.
- `chain(a, b, ...)` — yields items from each iterable in turn.
- `islice(it, stop)` or `islice(it, start, stop, step)` — slices a lazy sequence like a list slice.

`count` and `cycle` are infinite; consume them with `islice`, or `list()` will raise `MemoryError`.

Problem 24. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('islice=', list(itertools.islice(itertools.count(0, 2), 5)))
    print('sum=', sum(itertools.islice(itertools.count(1, 3), 4)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 25. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('islice=', list(itertools.islice(itertools.count(1, 2), 5)))
    print('sum=', sum(itertools.islice(itertools.count(0, 3), 4)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 26. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('cycle=', list(itertools.islice(itertools.cycle([1, 2, 3]), 7)))
    print('sum=', sum(itertools.islice(itertools.cycle([4, 5]), 5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 27. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    it = itertools.chain([1, 2, 3], [4, 5, 6])
    print('first=', list(itertools.islice(it, 4)))
    print('rest=', list(it))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 28. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('start')
    print('list=', list(itertools.cycle([1, 2, 3])))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

2.4 Pipelines

Individual lazy tools compose into larger lazy pipelines. Each stage pulls from the stage before it, and `islice` or `take` terminates an otherwise infinite chain.
Common shapes: `islice(map(f, xs), n)`, `sum(islice(chain(a, b), n))`, `take(filter(p, xs), n)`. The same principles apply no matter how many stages the pipeline has.

Problem 29. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
try:
    print('sum=', sum(itertools.islice(map(lambda x: x * x, fib()), 5)))
    print('list=', list(itertools.islice(map(lambda x: 2 * x, fib()), 5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 30. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
try:
    print('sum=', sum(itertools.islice(map(lambda x: x * x, fib()), 4)))
    print('list=', list(itertools.islice(map(lambda x: 2 * x, fib()), 6)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 31. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('a=', list(itertools.islice(itertools.chain([1, 2, 3], itertools.count(0)), 6)))
    print('b=', sum(itertools.islice(itertools.chain([1, 2, 3], itertools.count(0)), 5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 32. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('evens=', list(itertools.islice(filter(lambda x: x % 2 == 0, nats()), 5)))
    print('threes=', list(itertools.islice(filter(lambda x: x % 3 == 0, nats()), 5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 33. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    print('a=', list(take(map(lambda x: x * x, [1, 2, 3, 4]), 2)))
    print('b=', sum(take(map(lambda x: x + 1, [1, 2, 3, 4]), 3)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 34. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('start')
    print('list=', list(filter(lambda x: True, nats())))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

3. Desugaring and Dunders

Python's `for x in c` loop is sugar for calling `iter(c)` once, then `next()` repeatedly until `StopIteration`. A generator is a specialized iterator; a class can be its own iterator by defining `__iter__` and `__next__`.

3.1 Hand-written Countdown Class

A class that models a countdown can implement the iterator protocol directly:

- `__iter__` returns `self` (or a fresh iterator instance) so the class is iterable.
- `__next__` returns the next value or raises `StopIteration`.

If `__iter__` returns `self`, the class is one-shot like a generator: consuming it a second time gives an empty pass. If `__iter__` returns a fresh iterator each call, the class is re-iterable.

Problem 35. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    print('list=', list(Countdown(3)))
    print('sum=', sum(Countdown(3)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 36. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        self.i -= 1
        return self.i + 1
try:
    print('start')
    print('list=', list(Countdown(3)))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 37. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i
try:
    print('list=', list(Countdown(3)))
    print('sum=', sum(Countdown(3)))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 38. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class CountdownIter:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1

class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return CountdownIter(self.n)
try:
    c = Countdown(3)
    print('a=', list(c), 'b=', list(c))
    print('c=', list(c), 'sum=', sum(c))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 39. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    c = Countdown(3)
    print('first=', list(c))
    print('second=', list(c))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

3.2 For-loop Desugaring

`for x in c: body` is equivalent to:

```
it = iter(c)
while True:
    try:
        x = next(it)
    except StopIteration:
        break
    body
```

This explains why `for` works on generators, lists, files, and instances of user classes alike: they all supply an `__iter__` method. It also explains why a one-shot iterator behaves differently from a re-iterable container when looped over twice.

Problem 40. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    c = Countdown(3)
    it = iter(c)
    print('next=', next(it))
    print('rest=', list(it))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 41. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i

class CountdownIter:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1

class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return CountdownIter(self.n)
try:
    c1 = countdown(3)
    print('iter(c1) is c1:', iter(c1) is c1)
    c2 = Countdown(3)
    print('iter(c2) is c2:', iter(c2) is c2)
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 42. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class CountdownIter:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1

class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return CountdownIter(self.n)
try:
    c = Countdown(3)
    print('sum1=', sum(c))
    print('sum2=', sum(c))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 43. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    c = Countdown(3)
    print('for=', [x for x in c])
    print('next after for=', next(c))
except StopIteration:
    print('StopIteration')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

4. Real-world Examples

Previous sections introduced the tools; this section applies them to familiar data structures.

4.1 Range

Python's built-in `range` is re-iterable. A class-based `Range` with a generator-returning `__iter__` matches that behavior: each call to `iter(c)` produces a fresh generator, so the container can be looped over many times. Contrast with a class that stores a single generator instance: that class is one-shot, like the generator itself.

Problem 44. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step
    def __iter__(self):
        i = self.start
        while i < self.stop:
            yield i
            i += self.step
try:
    print('list=', list(Range(0, 5)))
    print('sum=', sum(Range(0, 5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 45. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step
    def __iter__(self):
        i = self.start
        while i <= self.stop:
            yield i
            i += self.step
try:
    print('list=', list(Range(0, 5)))
    print('sum=', sum(Range(0, 5)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 46. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step
    def __iter__(self):
        i = self.start
        while i < self.stop:
            yield i
            i += self.step + 1
try:
    print('list=', list(Range(0, 10, 3)))
    print('sum=', sum(Range(0, 10, 3)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 12 / 16 = 0.75

Problem 47. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step
    def __iter__(self):
        i = self.start
        while i < self.stop:
            yield i
            i += self.step

class RangeGen:
    def __init__(self, start, stop, step=1):
        self.gen = (i for i in range(start, stop, step))
    def __iter__(self):
        return self.gen
try:
    r1 = Range(0, 4)
    print('r1 twice=', list(r1), list(r1))
    r2 = RangeGen(0, 4)
    print('r2 twice=', list(r2), list(r2))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 48. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i

class Range:
    def __init__(self, start, stop):
        self.start = start
        self.stop = stop
    def __iter__(self):
        i = self.start
        while i < self.stop:
            yield i
            i += 1
try:
    g = countdown(5)
    print('g: sum=', sum(g), 'then list=', list(g))
    r = Range(0, 5)
    print('r: sum=', sum(r), 'then list=', list(r))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 49. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Range:
    def __init__(self, start, stop, step=1):
        self.start = start
        self.stop = stop
        self.step = step
    def __iter__(self):
        i = self.start
        while i < self.stop:
            return i
            i += self.step
try:
    print('start')
    print('list=', list(Range(0, 10, 0)))
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

4.2 BST Traversals

A binary search tree can expose its in-order, pre-order, or post-order traversal as a generator. `yield from` recursively delegates to subtree traversals, so the shape of the tree determines the output order.
Because the traversal is a generator method (not a stored generator instance), the tree is re-iterable: two successive `list(t)` calls each start a fresh traversal.
A skewed insert order (all values decreasing) produces a tree that is a single path; the in-order traversal is still sorted, but the pre-order traversal reveals the degenerate shape.

Problem 50. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return Node(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def inorder(t):
    if t is None:
        return
    yield from inorder(t.left)
    yield t.val
    yield from inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None
    def insert(self, val):
        self.root = insert(self.root, val)
    def __iter__(self):
        return inorder(self.root)
try:
    t = Tree()
    for v in [5, 3, 8, 1, 4]:
        t.insert(v)
    xs = list(t)
    print('first3=', xs[:3])
    print('last2=', xs[-2:])
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 51. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return Node(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def preorder(t):
    if t is None:
        return
    yield t.val
    yield from preorder(t.left)
    yield from preorder(t.right)

class Tree:
    def __init__(self):
        self.root = None
    def insert(self, val):
        self.root = insert(self.root, val)
    def __iter__(self):
        return preorder(self.root)
try:
    t = Tree()
    for v in [5, 3, 8, 1, 4]:
        t.insert(v)
    xs = list(t)
    print('first3=', xs[:3])
    print('last2=', xs[-2:])
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 52. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return Node(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def inorder(t):
    if t is None:
        return
    yield from inorder(t.left)
    yield t.val
    yield from inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None
    def insert(self, val):
        self.root = insert(self.root, val)
    def __iter__(self):
        return inorder(self.root)
try:
    t = Tree()
    for v in [5, 3, 8, 1, 4]:
        t.insert(v)
    print('a=', list(t))
    print('b=', list(t))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 53. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return Node(val)
    if val < root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def inorder(t):
    if t is None:
        return
    yield from inorder(t.left)
    yield t.val
    yield from inorder(t.right)

def preorder(t):
    if t is None:
        return
    yield t.val
    yield from preorder(t.left)
    yield from preorder(t.right)

class Tree:
    def __init__(self):
        self.root = None
    def insert(self, val):
        self.root = insert(self.root, val)
    def __iter__(self):
        return inorder(self.root)
try:
    t = Tree()
    for v in [5, 4, 3, 2, 1]:
        t.insert(v)
    print('inorder=', list(inorder(t.root)))
    print('preorder=', list(preorder(t.root)))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 54. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if root is None:
        return Node(val)
    if val <= root.val:
        root.left = insert(root.left, val)
    elif val > root.val:
        root.right = insert(root.right, val)
    return root

def inorder(t):
    if t is None:
        return
    yield from inorder(t.left)
    yield t.val
    yield from inorder(t.right)

class Tree:
    def __init__(self):
        self.root = None
    def insert(self, val):
        self.root = insert(self.root, val)
    def __iter__(self):
        return inorder(self.root)
try:
    t = Tree()
    for v in [5, 3, 3]:
        t.insert(v)
    xs = list(t)
    print('list=', xs)
    print('len=', len(xs))
except TypeError:
    print('TypeError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

LLM Model Performance

